

52. Blob

`ArrayBuffer` aur views JS (ECMA standard) ka hissa hain. Browser me unke upar ke high-level objects **File API** me hain, jaise `Blob`.

`Blob` = optional string `type` (aam taur par MIME type) + `blobParts` (dusre Blobs, strings aur `BufferSource` ka sequence).

Banana

js

```
new Blob(blobParts, options);
```

- `blobParts`: array of `Blob`/`BufferSource`/`String`
- `options`:
 - `type`: jaise `image/png`
 - `endings`: line endings badalna (`"transparent"` default, ya `"native"`)

js

```
let blob = new Blob(["<html>...</html>"], {type: 'text/html'}); // pehla argume  
  
let hello = new Uint8Array([72, 101, 108, 108, 111]);  
let blob2 = new Blob([hello, ' ', 'world'], {type: 'text/plain'});
```

Slice

js

```
blob.slice([byteStart], [byteEnd], [contentType]);
```

`array.slice` jaisa (negative numbers bhi chalte hain). **Blobs immutable** hote hain (string ki tarah): badal nahi sakte, sirf slice karke naye Blobs bana sakte ho.

Blob ko URL banana

Blob ko `<a>`, `<img>` ke URL ki tarah use kar sakte ho. `type` network requests me `Content-Type` ban jaata hai.

Download link:

html

```
<a download="hello.txt" href="#" id="link">Download</a>
<script>
let blob = new Blob(["Hello, world!"], {type: 'text/plain'});
link.href = URL.createObjectURL(blob);
</script>
```

`download` attribute browser ko navigate karne ke bajay download karne par majboor karta hai.

Bina HTML ke JS se:

```
js

let link = document.createElement('a');
link.download = 'hello.txt';
let blob = new Blob(['Hello, world!'], {type: 'text/plain'});
link.href = URL.createObjectURL(blob);
link.click();
URL.revokeObjectURL(link.href);
```

`URL.createObjectURL(blob)` ek unique URL banata hai: `blob:<origin>/<uuid>`. Browser andar URL -> Blob mapping rakhta hai, isliye URL chhota par Blob tak pahunch deta hai.

- URL sirf **current document** ke khule rehne tak valid hai.
- **Side effect:** jab tak mapping hai, Blob **memory me** rehta hai (free nahi ho sakta). Document unload par mapping saaf hoti hai, par lambe chalne wale app me nahi.
- Isliye zarurat khatam hone par `URL.revokeObjectURL(url)` call karo (mapping hat jaati hai, memory free ho sakti hai). Iske baad URL kaam nahi karta, to jab tak URL chahiye tab tak revoke mat karo.

Blob ko base64 me badalna

Alternative: **data URL** (`data:[<mediatype>][;base64],<data>`), jise har jagah normal URL ki tarah use kar sakte hain. Iske liye `FileReader`:

```
js

let reader = new FileReader();
reader.readAsDataURL(blob);
reader.onload = function() {
  link.href = reader.result; // data url
  link.click();
};
```

URL.createObjectURL(blob)

Data URL

Revoke karna padta hai (memory ke liye)

Kuch revoke nahi karna

Blob seedha access, encode/decode nahi

Bade Blobs par encoding se performance/memory nuksaan

Zyaadatar `createObjectURL` simple aur tez hai.

Image se Blob

`<canvas>` se:

1. `canvas.drawImage` se image (ya uska hissa) draw karo.
2. `canvas.toBlob(callback, format, quality)` call karo (async).

js

```
let canvas = document.createElement('canvas');
canvas.width = img.clientWidth;
canvas.height = img.clientHeight;
canvas.getContext('2d').drawImage(img, 0, 0);

canvas.toBlob(function(blob) {
  let link = document.createElement('a');
  link.download = 'example.png';
  link.href = URL.createObjectURL(blob);
  link.click();
  URL.revokeObjectURL(link.href);
}, 'image/png');
```

`async/await` ke saath:

js

```
let blob = await new Promise(resolve => canvasElem.toBlob(resolve, 'image/png'))
```

Page ka screenshot: `html2canvas` library (page ko canvas par draw karti hai), phir `toBlob`.

Blob se ArrayBuffer

js

```
const buffer = await blob.arrayBuffer();
```

Blob se stream

2GB se bade blobs par `arrayBuffer` bahut memory leta hai, to `blob.stream()` (ek `ReadableStream`) se tukdon me padho:

```
js

const stream = blob.stream().getReader();
while (true) {
  let { done, value } = await stream.read();
  if (done) break;
  console.log(value); // blob ka agla hissa
}
```

Summary

- `ArrayBuffer`/`Uint8Array` = binary data, **Blob** = type ke saath binary data. Upload/download ke liye convenient.
- `fetch`, `XMLHttpRequest` Blob ko natively samajhte hain.
- Typed array -> Blob: `new Blob(...)`. Blob -> ArrayBuffer: `blob.arrayBuffer()`.